Persistent

gitlab Runner

by Vishwanath MS

GitLab Runner Deep Dive – Overview

This section provides a high-level introduction to GitLab Runners, explaining their purpose, importance in CI/CD, and fundamental concepts.

1 2 3

What is a GitLab Runner?

- A GitLab Runner is an agent that executes your CI/CD jobs within a GitLab pipeline.
- It connects your GitLab instance to the infrastructure where your code is built, tested, and deployed.
- It effectively acts as the workhorse for your automation tasks.

Why are Runners Important in CI/CD?

Runners are crucial because they perform the actual work defined in your `.gitlab-ci.yml` file.

- They provide the necessary execution environment.
- They enable fast, consistent, and automated delivery of your software without manual intervention.

Key Concepts

Jobs: The smallest units of work in a pipeline, defining what to do.

Pipelines: A sequence of jobs that orchestrate your entire CI/CD workflow.

Executors: The method a Runner uses to run a job, such as Docker, Shell, or Kubernetes.

What is a GitLab Runner?

Open-Source Application

A GitLab Runner is an open-source application designed to work seamlessly with GitLab CI/CD pipelines.

Job Execution

It is responsible for executing the CI/CD jobs defined in your `.gitlab-ci.yml` file.

GitLab Communication

The Runner securely communicates with your GitLab server to fetch job details and send back results.

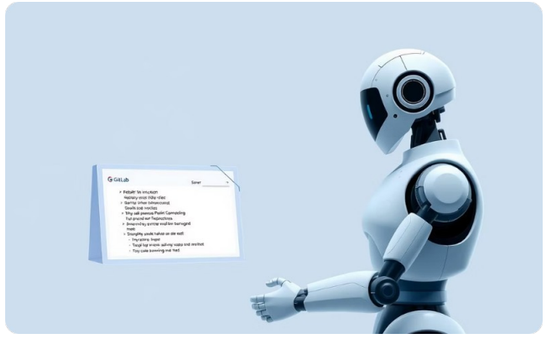
Flexible Infrastructure

It can be installed and run on diverse infrastructures, including local machines, virtual machines, cloud instances, and containers.

Runner Architecture & Workflow

Runner Registration

The GitLab Runner establishes a secure, persistent connection with the GitLab instance for job communication.

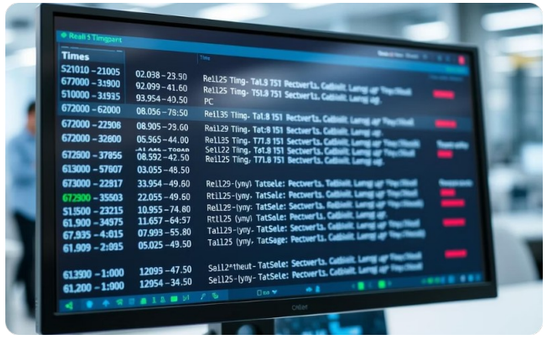


Job Assignment

GitLab assigns pending CI/CD jobs to the Runner, sending detailed instructions from the `.gitlab-ci.yml` file.

Environment Launch

The Runner's configured executor provisions the necessary environment (e.g., Docker, Shell, Kubernetes) to run the assigned job.



Execution & Reporting

The job's scripts are executed, with logs streamed and the final status (success/failure) reported back to GitLab.

Persistent

Types of GitLab Runners

Shared Runners

- Available to all projects within a GitLab instance.
- Managed by the GitLab administrator.
- Offer a convenient, 'out-of-the-box' solution for many teams.
- Suitable for projects that don't require specialized environments or strict isolation.

Specific Runners

- Dedicated to a particular project or group of projects.
- Provide greater control over the execution environment, dependencies, and security.
- Ideal for sensitive projects or those with unique requirements.

Group Runners

- Registered at the group level.
- Available for all projects within that group and its subgroups.
- Balance flexibility of Specific Runners with shared access of Shared Runners.
- Simplify management for teams working on multiple related projects.

Shell Executor

- **Direct Execution**

  The Shell Executor runs CI/CD jobs directly on the local shell environment of the machine where the Runner is installed, leveraging its existing software and configurations.

- **Non-Containerized Builds**

  This executor is well-suited for projects that do not rely on containerized environments for their build processes, or when you need direct access to host system resources.

- **Setup & Debugging**

  It is relatively simple to set up, requiring minimal configuration. Debugging issues can also be straightforward as you're working directly on the host machine.

- **Isolation & Pollution**

  A key limitation is the lack of strong job isolation. There's a potential for "pollution" where artifacts or environmental changes from one job can affect subsequent jobs on the same Runner.

Docker Executor

Clean & Isolated

Each job runs in a brand-new, temporary Docker container, ensuring strong isolation and preventing interference between jobs.

Reproducible Environments

Docker guarantees consistent build and test environments, leading to reliable and predictable code behavior across runs.

Flexible Image Selection

Easily choose different Docker images for each job in your `.gitlab-ci.yml`, offering precise control over tools and dependencies.

Advanced Configurability

Supports advanced settings like Docker-in-Docker for complex builds and custom Docker images for managing intricate dependencies.

Persistent

Kubernetes Executor

Container Orchestration

The Kubernetes Executor schedules each CI/CD job as a dedicated Pod within your Kubernetes cluster, leveraging its powerful orchestration and resource management capabilities.

Dynamic Scalability

Provides exceptional scalability by dynamically provisioning environments on demand. Jobs run in isolated, temporary Pods, ensuring a clean and consistent state for every execution.

Cloud-Native Integration

Perfect for larger teams and cloud-native organizations, as it seamlessly integrates CI/CD workflows with existing Kubernetes infrastructures, optimizing resource utilization.

Comparing Executors

Shell Executor

Executes jobs directly on the host machine. Offers **low isolation** and is best for **simple local builds** due to its **low setup complexity**.

Docker Executor

Runs each job in a dedicated Docker container. Provides **high isolation** and is the **most common** choice for consistent environments with **medium setup complexity**.

Kubernetes Executor

Schedules each job as a Pod within a Kubernetes cluster. Offers **very high isolation** and is ideal for **large, scalable cloud-native use cases** despite its **high setup complexity**.

Registering a Runner

Install Runner Binary

Begin by downloading and installing the `gitlab-runner` executable on your chosen host machine (e.g., Linux, Windows, macOS).

Run Registration Command

Use the command line interface to initiate the registration process:

```
gitlab-runner register
```

Configure Runner

You will be prompted to provide essential information for connecting your Runner to GitLab:

- GitLab instance URL
- Registration token (from your project, group, or instance settings)
- Executor type (e.g., Shell, Docker, Kubernetes)
- A descriptive name and relevant tags for categorization

Runner Registration Demo (Shell Example)

To register a GitLab Runner, you primarily use the command-line interface. This demo focuses on the Shell executor, demonstrating the steps to connect your local machine as a Runner.

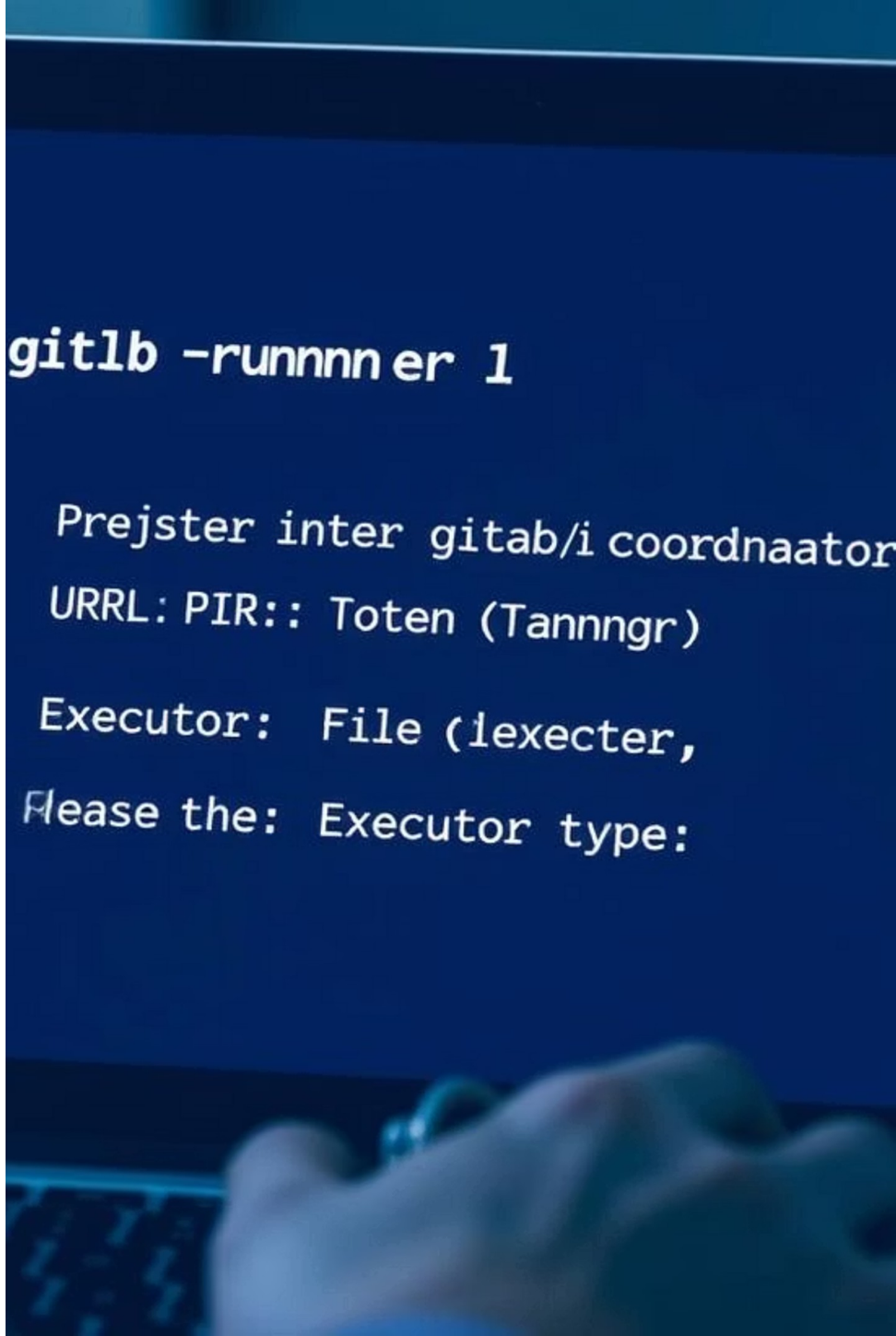
```
sudo gitlab-runner register
```

The command-line interface will prompt you for the following essential details to complete the registration process:

Enter your GitLab instance URL (e.g., `https://gitlab.com/` or your self-hosted instance URL)

- Enter the registration token (found in your GitLab project, group, or instance CI/CD settings)
- Provide a descriptive name for your Runner and relevant tags for categorization

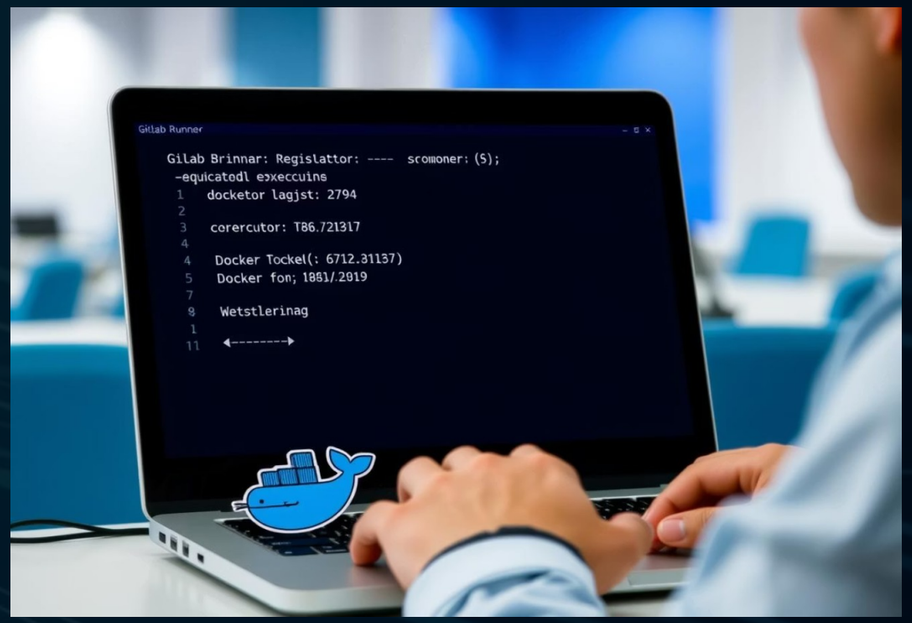
Select the executor type (e.g., `shell`, `docker`, `kubernetes`)



Registering a Docker Runner – Example

To register a GitLab Runner configured to use the Docker executor, you can provide all necessary parameters directly in the registration command, avoiding interactive prompts.

```
sudo gitlab-runner register \  --url https://gitlab.com/ \  --registration-token <TOKEN> \  --executor docker \  --description "docker-runner" \  --docker-image "alpine:latest"
```



This command specifies:

`--executor docker`: Instructs the Runner to use Docker for job execution, ensuring isolation and reproducibility.

`--docker-image "alpine:latest"`: Sets the default Docker image (`alpine:latest` in this case) that jobs will use if no specific image is defined in the `.gitlab-ci.yml` file.

Registering a Kubernetes Runner – Example

Deploying a GitLab Runner configured to use the Kubernetes executor is typically managed through Helm, the package manager for Kubernetes. Helm streamlines the process of defining, installing, and upgrading applications within your cluster.

```
helm repo add gitlab https://charts.gitlab.iohelm install gitlab-runner gitlab/gitlab-runner \  --set gitlabUrl=https://gitlab.com/ \  --set runnerRegistrationToken=<TOKEN>
```

This Helm command:

Adds the GitLab Helm repository: `helm repo add gitlab https://charts.gitlab.io` registers the official GitLab charts.

Installs the GitLab Runner: `helm install gitlab-runner gitlab/gitlab-runner` deploys the Runner as a set of Kubernetes resources.

Sets GitLab URL and Registration Token: The `--set` flags pass crucial configuration like your GitLab instance URL and the Runner's registration token, automating the connection to your GitLab project or group.

Runner Visibility & Management

Once registered, GitLab Runners are visible and manageable directly within the GitLab UI, providing centralized control over your CI/CD infrastructure.

Check Registered Runners

Navigate to **Project > Settings > CI/CD > Runners** to view a comprehensive list of all registered Runners associated with your project, group, or instance.

Manage Status & Tags

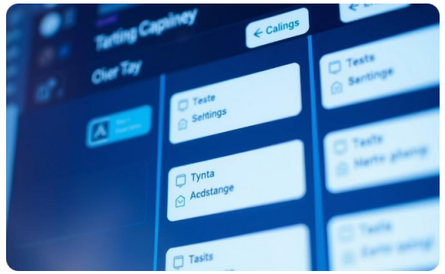
From the UI, you can easily enable or disable Runners, assign descriptive tags for better job routing, and check their current status (online/offline).

Security Best Practices

Regularly review active Runners. Unregister unused Runners, and ensure sensitive tokens are securely managed. Avoid running untrusted code on Runners with broad permissions.

Best Practices for GitLab Runners

Optimizing your GitLab Runner setup is crucial for efficient, secure, and scalable CI/CD pipelines. Adhering to best practices ensures reliability and performance.



Tag Runners for Job Routing

Assign descriptive tags (e.g., `docker`, `frontend`, `production`) to your Runners. This allows precise control over which jobs run on specific Runners, ensuring compatibility and resource allocation.

Use Isolation for Untrusted Code

For jobs executing untrusted or potentially malicious code, always use Executors providing strong isolation, such as Docker or Kubernetes. Shell executors should be reserved for trusted environments only.

Limit Shared Runner Use

While convenient, shared Runners on gitlab.com might not be suitable for sensitive data or specific resource needs. Consider private or specific Runners for such cases.

Best Practices for GitLab Runners...

Regularly Update Runners Keep your GitLab Runner installations up-to-date. Regular updates bring performance improvements, bug fixes, and critical...

Best Practices for GitLab Runners (Continue

Regularly Update Runners

Keep your GitLab Runner installations up-to-date. Regular updates bring performance improvements, bug fixes, and critical security patches, protecting your CI/CD environment.

Monitor Runner Resource Usage

Implement monitoring for your Runners to track CPU, memory, and disk usage. This helps identify bottlenecks, optimize resource allocation, and prevent job failures due to insufficient resources.

Persistent

Troubleshooting & Tips

Efficiently manage and troubleshoot your GitLab Runners with these essential tips and commands.

Common Errors & Logs

For most issues, start by checking the Runner logs, typically located at `/var/log/gitlab-runner/` on Linux systems. Look for error messages related to connectivity, permissions, or executor issues.

Useful CLI Commands

`gitlab-runner list`: View all registered Runners on the current machine.

`gitlab-runner verify`: Check if Runners are still connected to GitLab and valid.

`gitlab-runner restart`: Restart the Runner service after configuration changes.

For large-scale deployments, consider **Runner autoscaling** with Docker Machine or Kubernetes. This allows Runners to dynamically provision or de-provision compute resources based on job queue demand, optimizing cost and performance.

Key Takeaways & Further Resources

CI/CD Backbone

GitLab Runners are fundamental to automating your CI/CD pipelines, executing jobs directly on your infrastructure.

Strategic Executor Choice

Carefully select the appropriate executor (Shell, Docker, Kubernetes) based on your team size, project needs, and desired isolation levels.

Security & Maintenance

Prioritize keeping your Runners secure with proper permissions, and ensure they are regularly updated for optimal performance and protection.

For in-depth information, refer to the Official GitLab Runner Documentation.